

Universitatea POLITEHNICA din București
Facultatea de Automatică și Calculatoare

Proiectare Arhitecturală

RXP Custom 3D

Magazin e-commerce pentru accesorii auto tuning cu imprimare 3D

Grupa: 332CA | **Seria:** CA | **Data:** Aprilie 2026 | **Versiune:** 1.0

Cuprins

1. Introducere	3
1.1. Scopul sistemului	3
1.2. Definiții, acronime	3
1.3. Documente referințe	3
2. Obiective de proiectare	4
2.1. Obiective funcționale	4
2.2. Obiective de calitate (nefuncționale)	4
2.3. Constrângeri și principii arhitecturale	4
3. Arhitectura propusă	5
3.1. Decompoziția în subsisteme	5
3.2. Distribuția pe platforme hardware/software	8
3.3. Managementul datelor persistente	9
3.4. Controlul accesului utilizatorilor la sistem	11
3.5. Fluxul global al controlului	12
3.6. Tratarea condițiilor limită	13
Glosar de termeni	14

1. Introducere

1.1. Scopul sistemului

Prezentul document descrie arhitectura software a platformei **RXP Custom 3D**, un magazin online B2C specializat în accesorii auto de tip tuning, cu accent pe produse fabricate prin imprimare 3D la comandă. Documentul definește decompoziția în subsisteme, distribuția lor pe platforme hardware/software, managementul datelor persistente, controlul accesului, fluxul global al controlului și tratarea condițiilor limită. Scopul este de a oferi echipei de dezvoltare o referință tehnică completă care transpune cerințele funcționale și nefuncționale într-o soluție implementabilă, trasabilă și verificabilă.

Arhitectura este de tip **multi-tier containerizat** (client-server în trei niveluri: prezentare, logică de business, date), implementată ca microservicii coordonate prin Docker Compose. Această abordare asigură separarea responsabilităților, scalabilitate orizontală selectivă și portabilitate între medii de execuție (dev, staging, prod).

1.2. Definiții, acronime

Termen	Definiție
ADD	Architectural Design Document — prezentul document
API	Application Programming Interface — interfață REST expusă de backend
CI/CD	Continuous Integration / Continuous Deployment
COD	Cash On Delivery — plată la livrare (ramburs)
CORS	Cross-Origin Resource Sharing
DTO	Data Transfer Object — schemă Pydantic pentru serializare
GDPR	Regulamentul UE 2016/679 privind protecția datelor
HTTPS	HTTP peste TLS 1.2+ (port 443)
JWT	JSON Web Token — transmitere securizată a sesiunilor
ORM	Object-Relational Mapping (SQLAlchemy)
RBAC	Role-Based Access Control — control acces bazat pe roluri
REST	Representational State Transfer — stil arhitectural API
SKU	Stock Keeping Unit — cod unic produs
SRS	Software Requirements Specification (Specificația Cerințelor)
TLS	Transport Layer Security — criptare canal de comunicație
TVA	Taxa pe Valoarea Adăugată
UC	Use Case — caz de utilizare
WAL	Write-Ahead Log (PostgreSQL, mecanism de durabilitate)

1.3. Documente referințe

Cod	Document / Standard
SRS-RXP	Documentul de specificație a cerințelor — RXP Custom 3D, Martie 2026
ISO/IEC/IEEE 42010:2011	Systems and software engineering — Architecture description
RFC 7519	JSON Web Token (JWT)
RFC 8446	TLS 1.3 — Transport Layer Security
OWASP ASVS 4.0	Application Security Verification Standard
Legea 571/2003	Codul fiscal al României (facturare și TVA)
Regulamentul UE 2016/679	GDPR
FastAPI 0.11x	Documentație oficială framework backend
PostgreSQL 15	Documentație oficială sistem de baze de date
Stripe API 2024-06-20	Documentație oficială procesator plăți

2. Obiective de proiectare

Obiectivele de proiectare derivă direct din cerințele funcționale (CF-01...CF-12) și nefuncționale (CNF-01...CNF-06) definite în SRS-RXP. Acestea ghidează deciziile arhitecturale majore și servesc drept criterii de acceptanță pentru validarea arhitecturii.

2.1. Obiective funcționale

Obiectiv	Descriere	Trasabilitate SRS
OF-01	Suport complet pentru ciclul de viață al unei comenzi: catalog, coș, checkout, plată, factură	CF-01...CF-07
OF-02	Canal dedicat pentru cereri de printare 3D personalizată cu upload STL/OBJ/STEP	CF-08
OF-03	Panou administrativ pentru gestiune produse, comenzi și setări firmă	CF-10, CF-11, CF-12
OF-04	Suport dual pentru plată la livrare (COD) și card bancar prin Stripe	CF-05, CF-06
OF-05	Generare automată factură PDF conformă cu legislația românească	CF-07

2.2. Obiective de calitate (nefuncționale)

Obiectiv	Descriere / criteriu măsurabil	Trasabilitate
OC-01	Securitate: JWT HttpOnly, bcrypt cost ≥ 12 , HTTPS obligatoriu, whitelist extensii upload	CNF-01
OC-02	Performanță: răspuns API < 500 ms (p95), tranzacții de stoc cu lock pesimist	CNF-02
OC-03	Disponibilitate: uptime 99.5%, retry DB la startup (max 10 încercări)	CNF-03
OC-04	Conformitate legală: GDPR, Legea 571/2003, TVA explicit pe factură și produs	CNF-04
OC-05	Scalabilitate: Docker Compose, PostgreSQL separat de API, servicii independente	CNF-05
OC-06	Compatibilitate: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+, UI responsive	CNF-06
OC-07	Mentenabilitate: separație pe router FastAPI, ORM SQLAlchemy, schemă versionată	Best practice

2.3. Constrângeri și principii arhitecturale

Proiectarea respectă următoarele principii, în ordinea priorității:

- **Separația responsabilităților** — fiecare subsistem are un rol bine definit (prezentare, logică, persistență, securitate periferică);
- **Stateless API** — backend-ul nu menține sesiune în memorie; starea este în JWT (client) și baza de date;
- **Single Source of Truth** — PostgreSQL este singura sursă canonică pentru entități de business (produse, comenzi, utilizatori);

- **Fail-safe defaults** — by default accesul este refuzat; endpoint-urile expun explicit ce roluri sunt acceptate;
- **Idempotență la webhook-uri** — procesarea Stripe se bazează pe `payment_intent_id` unic, duplicate sunt ignorate;
- **Observabilitate** — logging structurat, health-check endpoint, metrice disponibile pentru monitoring extern;
- **Portabilitate** — Docker Compose asigură paritatea dev/prod și deployment pe orice host Linux.

3. Arhitectura propusă

Arhitectura RXP Custom 3D este organizată în **6 subsisteme** independente, fiecare rulat într-un container Docker dedicat sau integrat prin protocoale standardizate. Comunicarea internă se face prin rețeaua Docker virtuală, iar expunerea externă se realizează exclusiv prin reverse proxy-ul Nginx (TLS terminator).

3.1. Decompoziția în subsisteme

ID	Subsistem	Tehnologie	Responsabilitate principală
SS-1	Frontend (Web UI)	HTML5, CSS3, JavaScript vanilla, Fetch API	Prezentare, interacțiune cu utilizatorul, validare client
SS-2	Backend API	FastAPI, Pydantic, SQLAlchemy (Python 3.11)	Logică de business, validare, autorizare, orchestrare
SS-3	Baza de date	PostgreSQL 15	Persistență tranzacțională (ACID), integritate referențială
SS-4	Procesor plăți	Stripe API (extern, integrat prin SDK)	Procesare carduri, webhook-uri evenimente plată
SS-5	Generator facturi	ReportLab (modul Python intern)	Generare PDF conform legislației fiscale RO
SS-6	Reverse proxy	Nginx 1.25, OpenSSL	Terminare TLS, routing, servire statice, rate limiting

3.1.1. Subsistemul Frontend (Web UI)

Tehnologie: HTML5, CSS3 (flexbox/grid), JavaScript vanilla (ES2020), Fetch API pentru comunicarea cu backend-ul. Nu se utilizează framework-uri SPA (React/Vue) pentru a minimiza footprint-ul și a elimina build-step-ul.

Componente: pagini statice (index.html, category.html, product.html, cart.html, checkout.html, account.html, custom.html, login.html, register.html), un modul central *js/api.js* (wrapper fetch cu gestiune token/eroare) și *script.js* (logică UI comună: navbar, badge coș, toast notifications).

Interfețe:

- **Ieșire** către SS-2: HTTP/HTTPS REST (JSON), cu credentials=include pentru cookie JWT;
- **Intrare** de la utilizator: evenimente DOM (click, submit, input);
- **Integrare** cu SS-4: Stripe.js (Stripe Elements) pentru securizarea datelor cardului (PCI-DSS SAQ-A).

Frontend-ul este servit static de Nginx (SS-6), fără procesare server-side; toată logica dinamică rulează în browser.

3.1.2. Subsistemul Backend (API)

Tehnologie: FastAPI (ASGI, Uvicorn), Pydantic v2 pentru schemele DTO, SQLAlchemy 2.x ORM, PyJWT pentru tokens, bcrypt pentru parole.

Pachete interne: *app.routers* (auth, products, cart, orders, payments, custom_requests, admin_panel) — grupare funcțională endpoint-uri; *app.models* — entități ORM; *app.schemas* — DTO Pydantic; *app.deps* — dependențe injectabile (sesiune DB, autentificare); *app.security* — primitive cripto (hash, JWT); *app.utils.invoice* — wrapper pentru SS-5.

Interfețe:

- **Intrare:** REST pe `/api/*`, documentat automat prin OpenAPI 3.1 la `/docs`;
- **Ieșire** către SS-3: SQL prin driver `psycopg2`, pool de conexiuni `SQLAlchemy`;
- **Ieșire** către SS-4: HTTPS REST la `api.stripe.com` (`PaymentIntent`, `Webhook`);
- **Intrare** de la SS-4: `webhook POST /api/payments/webhook` (semnătura `HMAC` verificată);
- **Ieșire** către SS-5: apel intra-proces Python (fără overhead de rețea).

3.1.3. Subsistemul Date (DB)

Tehnologie: PostgreSQL 15 (image oficială Docker), stocare pe volum persistent `pgdata`. Izolare tranzacțională implicită: `READ COMMITTED`; ridicată la `SERIALIZABLE` pentru checkout cu `SELECT ... FOR UPDATE`.

Interfață: port 5432 expus doar în rețeaua Docker internă (nu este accesibil din exterior); conexiune prin TCP cu user/parolă.

3.1.4. Subsistemul Plată (Stripe)

Serviciu SaaS extern. Integrarea respectă fluxul `PaymentIntent` cu 3D Secure: (1) backend-ul creează `PaymentIntent` și returnează `client_secret`; (2) frontend-ul confirmă plata prin Stripe Elements; (3) Stripe notifică backend-ul prin `webhook`; (4) backend-ul marchează comanda ca plătită și decrementează stocul.

3.1.5. Subsistemul Facturare (PDF)

Modul Python intern `app.utils.invoice`, bazat pe `ReportLab`, cu font `DejaVuSans` pentru diacritice. Fișierele PDF sunt stocate pe volum Docker `/invoices` și servite prin endpoint autentificat `/api/orders/{id}/invoice`. Calea este sanitizată împotriva `path traversal`.

3.1.6. Subsistemul Reverse Proxy (Nginx)

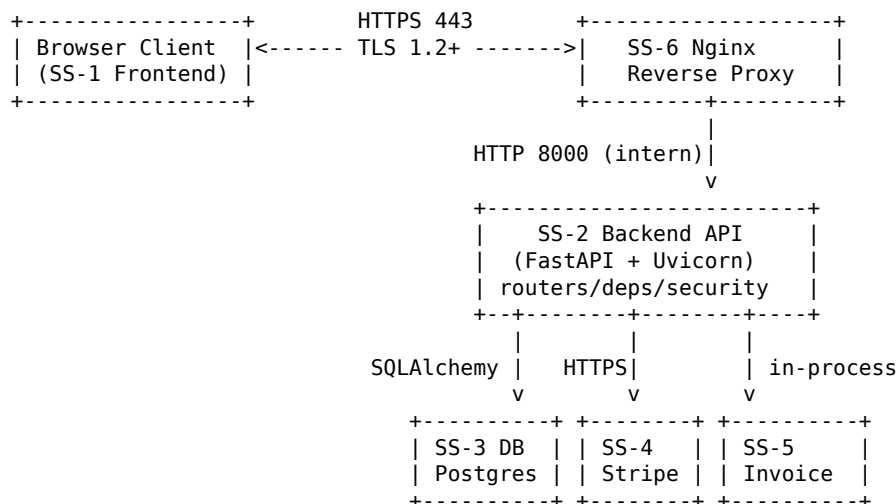
Tehnologie: Nginx 1.25 cu OpenSSL. Configurare în `nginx/conf.d/default.conf`.

Responsabilități:

- terminare TLS 1.2+ pe port 443, redirect 80 → 443;
- servire conținut static (`/`, `/static/`, `/images/`, `/js/`, frontend HTML);
- reverse proxy `/api/*` → backend:8000 (HTTP intern);
- headere de securitate: `HSTS`, `X-Content-Type-Options`, `X-Frame-Options`, `CSP`;
- rate limiting pe `/api/auth/login` (5 cereri/minut/IP) pentru mitigarea brute-force.

3.1.7. Diagrama de componente și interfețe

Diagrama de componente de nivel înalt (UML 2.5, interfețe provided / required):



Interfețele cheie sunt rezumate în tabelul următor:

Interfață	Furnizor	Consumator	Protocol / Format
I-Web	SS-6 Nginx	Browser utilizator	HTTPS 443 + HTML/CSS/JS
I-API	SS-2 Backend	SS-1 Frontend (via SS-6)	REST/JSON peste HTTPS
I-Db	SS-3 PostgreSQL	SS-2 Backend	TCP 5432 + SQL (SQLAlchemy)
I-Pay	SS-4 Stripe	SS-2 Backend	HTTPS REST (Stripe SDK)
I-Hook	SS-2 Backend	SS-4 Stripe	HTTPS POST cu semnătură HMAC
I-Inv	SS-5 Invoice	SS-2 Backend	Apel funcție Python (intra-proces)
I-Fs	Volum Docker	SS-2, SS-5	I/O disk (/uploads, /invoices)

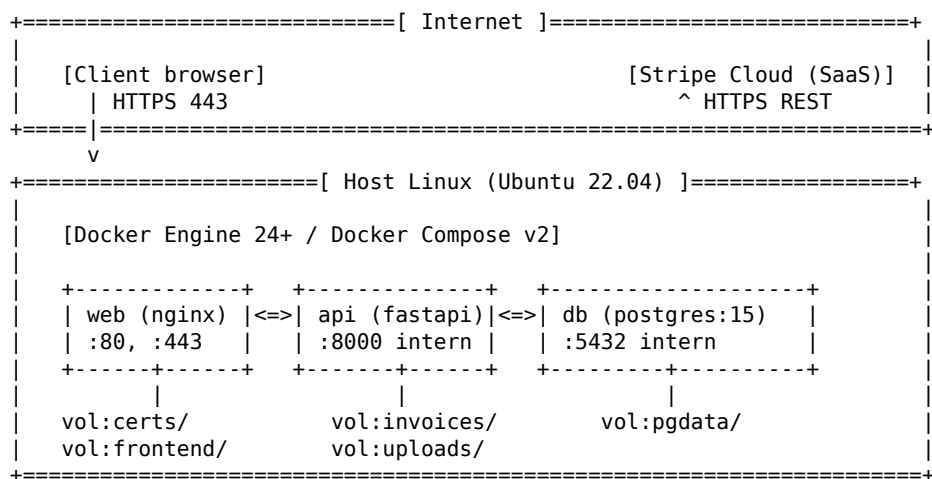
3.2. Distribuția pe platforme hardware/software

Sistemul este livrat ca o colecție de containere Docker orchestrate prin *docker-compose.yml*. Toate serviciile rulează pe un singur nod gazdă (single-host deployment) în configurația de bază, cu posibilitate de migrare la Docker Swarm / Kubernetes pentru scalare viitoare.

Specificații platforme de execuție

Nod	Hardware minim	Software	Servicii rulate
Server producție	2 vCPU, 2 GB RAM, 20 GB SSD	Ubuntu 22.04+, Docker 24+, Docker Compose v2	SS-2 (api), SS-3 (db), SS-6 (web)
Client (browser)	Oricărui device desktop/mobil	Chrome 90+, Firefox 88+, Safari 14+, Edge 90+	SS-1 (rulează în browser)
Stripe Cloud	N/A (SaaS)	Gestionat de Stripe Inc.	SS-4

Diagrama de distribuție (deployment)



Containere și volume

Container	Imagine de bază	Porturi expuse	Volume montate
web	nginx:1.25-alpine	80, 443 (host)	./nginx/conf.d, ./nginx/certs, ./frontend
api	python:3.11-slim	— (doar intern 8000)	./backend/app, invoices, uploads
db	postgres:15-alpine	— (doar intern 5432)	pgdata (named volume)

3.3. Managementul datelor persistente

Persistența este asigurată prin două mecanisme complementare: **bază de date relațională** PostgreSQL pentru entități de business și **sistem de fișiere** (volume Docker) pentru artefacte binare (imagini produse, fișiere 3D încărcate, facturi PDF).

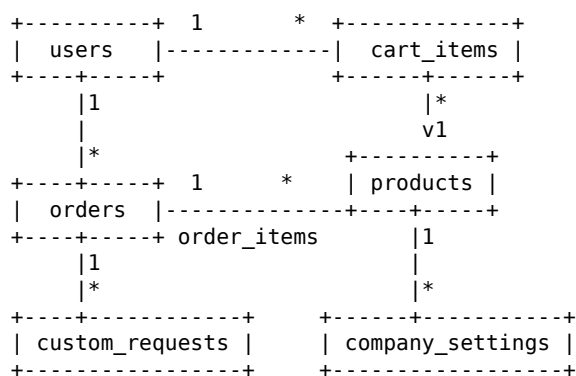
Sistemul de baze de date

PostgreSQL 15 — sistem ACID, cu WAL pentru durabilitate și replicare streaming disponibilă pentru scenarii HA. Schema este gestionată declarativ prin SQLAlchemy metadata (*create_all* la startup în dev; migrații Alembic recomandate pentru prod).

Schema conceptuală a bazei de date

Tabelă	Atribute cheie	Rol
users	id PK, email UNIQUE, password_hash, is_admin, created_at	Conturi utilizatori (Client/Admin)
products	id PK, sku UNIQUE, name, description, price (bani), currency, stock, is_active, image_url, category, tags[]	Catalog produse
cart_items	id PK, user_id FK, product_id FK, quantity	Coș persistent per utilizator
orders	id PK, user_id FK, total_amount (bani), currency, status, shipping_fee_minor, customer_name/phone/address, stripe_payment_intent, invoice_no UNIQUE, created_at	Antet comandă
order_items	id PK, order_id FK, product_id FK, quantity, unit_price (bani)	Linii comandă (snapshot preț)
custom_requests	id PK, user_id FK, file_path, file_format, description, status, created_at	Cereri printare 3D
company_settings	id PK (singleton=1), name, cif, reg_com, address, bank_account	Date firmă pentru facturare

Diagrama conceptuală (ER simplificat)



Convenții și constrângeri

- **Prețuri în unități minore** (bani) — evită erori de rotunjire floating-point; conversie la RON doar la afișare;
- **Chei străine cu ON DELETE RESTRICT** pentru comenzi; utilizatorii cu istoric nu pot fi șterși fără anonimizare prealabilă;
- **Index pe coloane critice:** products.sku, products.category, orders.status, orders.created_at, users.email;

- **Integritate tranzacțională** la checkout: *SELECT ... FOR UPDATE* pe produse înainte de *UPDATE stock* — elimină race condition;
- **Invoice numbering monotonic** — coloana *invoice_no* este *UNIQUE*, generată cu secvență Postgres pentru conformitate fiscală.

Fișiere (sisteme de fișiere pe volume Docker)

Director	Conținut	Acces	Backup
pgdata/	Date PostgreSQL (WAL + heap)	Numai container <i>db</i>	pg_dump zilnic (recomandat)
/uploads/custom/	Fișiere STL/OBJ/STEP (max 50 MB)	Citire: admin; Scriere: API prin CF-08	Snapshot volum săptămânal
/invoices/	Facturi PDF (r/w API, r autentificat utilizator)	API + admin	Snapshot volum săptămânal
/static/images/	Imagini produse (upload admin)	Citire publică	Git / snapshot volum

3.4. Controlul accesului utilizatorilor la sistem

Controlul accesului este implementat pe trei niveluri: **autentificare** (identificare utilizator), **autorizare** (verificare drepturi) și **protecție periferică** (Nginx, TLS, rate limiting).

Roluri și permisiuni

Rol	Autentificare	Permisiuni
Vizitator	Nu	Browse catalog, vizualizare detalii produs, căutare
Client	Da (JWT utilizator)	Toate cele ale Vizitatorului + coș, checkout COD/Stripe, cereri 3D, istoric comenzi personal, descărcare facturi proprii
Administrator	Da (JWT + <i>is_admin=true</i>)	Toate cele ale Clientului + CRUD produse, gestiune comenzi (status update), setări firmă, descărcare orice factură

Mecanism de autentificare

- **Parole:** hash bcrypt cu cost ≥ 12 , salt unic per utilizator; verificare cu *bcrypt.verify* timp constant;
- **JWT:** algoritm HS256, secret stocat în variabila de mediu *JWT_SECRET* (64+ octeți random); payload minim: *sub* (user_id), *exp* (7 zile), *iat*;
- **Transport:** cookie HttpOnly + Secure + SameSite=Lax, nume *auth_token*; fallback Bearer token pentru Swagger/clienți API;
- **Logout:** delete cookie client-side; token-ul rămâne valid până la *exp* (stateless); lista de revocare poate fi adoptată ulterior dacă e necesar.

Mecanism de autorizare

Autorizarea se face prin dependențe FastAPI injectate la nivel de endpoint:

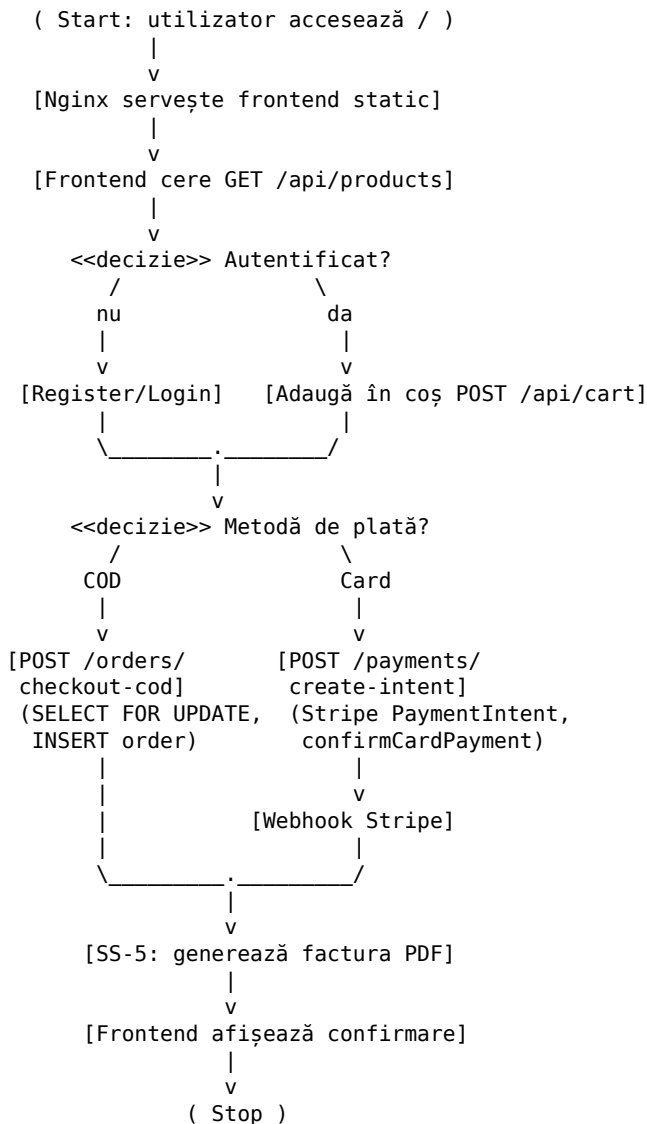
- *get_current_user_id* — citește JWT din cookie sau Bearer; 401 dacă lipsește sau e expirat;
- *admin_required* — extinde cele de mai sus, verifică *is_admin*; 403 dacă nu e administrator;
- **Ownership checks** — endpoint-urile de tip */orders/{id}* verifică *user_id == order.user_id* (sau admin) — previne IDOR.

Protecție periferică

Strat	Mecanism	Scop
Rețea	TLS 1.2+, HSTS (max-age 1an)	Confidențialitate și integritate în tranzit
Nginx	Rate limit 5 req/min pe /auth/login	Mitigare brute-force credentiale
Nginx	Headere CSP, X-Frame-Options, X-Content-Type-Options	Reducere suprafață XSS / clickjacking
API	CORS whitelist (<i>CORS_ORIGINS</i>)	Previne accesul din origini neautorizate
API	Validare Pydantic (tipuri, lungimi, pattern)	Prevenire injection și date malformate
Upload	Whitelist extensii (.stl, .obj, .step), limită 50 MB	Prevenire upload malware și DoS pe disk

3.5. Fluxul global al controlului

Diagrama de activitate de mai jos descrie fluxul principal end-to-end pentru un utilizator nou care achiziționează un produs (scenariu nominal): de la accesarea site-ului până la primirea facturii.



Comunicarea între subsisteme pe cazuri de utilizare

UC	Traseu control	Sincron / Asincron
UC-01 Auth	SS-1 → SS-6 → SS-2 → SS-3; răspuns JWT în cookie	Sincron
UC-02 Browse	SS-1 → SS-6 → SS-2 → SS-3 (SELECT paginat)	Sincron
UC-03 Cart	SS-1 → SS-2 → SS-3 (INSERT/UPDATE cart_items)	Sincron
UC-04 Checkout COD	SS-2 → SS-3 (tranz. SELECT FOR UPDATE) → SS-5 (PDF)	Sincron
UC-05 Checkout Stripe	SS-2 → SS-4 (PaymentIntent) → SS-1 (Elements); apoi SS-4 → SS-2 (webhook) → SS-3 → SS-5	Hibrid

UC	Traseu control	Sincron / Asincron
UC-06 Custom 3D	SS-1 → SS-2 (multipart) → FileSystem + SS-3	Sincron
UC-09 Factură	SS-2 → SS-5 (intra-proces) → FileSystem	Sincron

3.6. Tratarea condițiilor limită

Pornirea sistemului (startup)

- **docker compose up -d** pornește toate cele 3 containere în ordinea declarată (db → api → web) prin *depends_on*;
- **API retry DB**: la startup, SS-2 rulează un retry exponențial (max 10 încercări, delay crescător) până când PostgreSQL acceptă conexiuni;
- **Inactivare schemă**: dacă tabelele lipsesc, SQLAlchemy *create_all* le creează (dev); în prod se rulează *alembic upgrade head*;
- **Health check**: endpoint *GET /api/health* returnează 200 doar dacă SS-3 este accesibilă; Nginx returnează 503 cât timp API răspunde 5xx.

Oprirea sistemului (shutdown)

- **Graceful shutdown** la SIGTERM: Uvicorn așteaptă finalizarea cererilor active (timeout 30s) înainte de a închide worker-ele;
- **Tranzacțiile DB** în curs sunt fie finalizate, fie rollback-uite automat de PostgreSQL la închiderea conexiunii;
- **Volumele** persistă datele; PostgreSQL face checkpoint la shutdown ordonat pentru a minimiza replay WAL la repornire;
- **docker compose down** păstrează volumele; *docker compose down -v* **șterge** datele — rezervat doar pentru reset dev.

Tratarea condițiilor speciale

Condiție	Reacția sistemului
Stoc insuficient la checkout	<i>SELECT ... FOR UPDATE</i> detectează conflict; ROLLBACK; API răspunde 409 Conflict cu detaliu stoc disponibil
Card refuzat / fonduri insuficiente	Stripe returnează eroare în <i>confirmCardPayment</i> ; frontend afișează mesajul; comanda nu se creează (rely pe webhook)
Webhook Stripe duplicat	Verificare <i>stripe_payment_intent</i> UNIQUE în orders; a doua cerere devine no-op (idempotență)
Token JWT expirat	API returnează 401; frontend redirect la /login cu păstrarea paginii curente (?next=...)
Upload 3D peste limită	Nginx <i>client_max_body_size 60M</i> respinge; fallback API validează dimensiunea și returnează 413
Extensie upload neacceptată	API validează Content-Type și extensie; respinge cu 400 Bad Request
Pierdere conexiune DB în timpul cererii	SQLAlchemy event listener recrează conexiunea pool; cererea curentă eșuează cu 500; frontend reîncearcă (exponential backoff)
Eroare generare factură PDF	Comanda este creată oricum (nu blochează cumpărarea); eroarea este logată; job offline regenerează factura; admin poate descărca ulterior
Date firmă incomplete	Factura este generată cu mențiune <i>Date fiscale în curs de completare</i> ; admin notificat
CORS origin neautorizat	Middleware-ul FastAPI CORSMiddleware blochează cu 400

Cazuri de utilizare administrative

- **Backup DB:** comandă cron pe host — *docker exec db pg_dump ... > backup.sql*, retention 30 zile;
- **Reset parolă admin:** UPDATE direct în baza de date (hash bcrypt generat offline) — procedură documentată;
- **Rotire secret JWT:** modificare *JWT_SECRET* în env, restart API — invalidează toate sesiunile active (comportament așteptat);
- **Reînnoire certificate TLS:** Let's Encrypt cu certbot, hook post-renewal pentru *nginx -s reload*;
- **Monitorizare:** loguri Docker exportă către agregator (Loki/ELK); metrice Prometheus expuse de API la */metrics*.

Glosar de termeni

Termen	Explicație
ACID	Atomicitate, Consistență, Izolare, Durabilitate — proprietățile tranzacțiilor DB
ASGI	Asynchronous Server Gateway Interface — standardul Python pentru server web asincron
Backend	Componenta server-side a aplicației, responsabilă de logica de business și persistență
bcrypt	Algoritm de hash pentru parole, rezistent la atacuri brute-force prin cost configurabil
Container	Unitate de rulare izolată Docker, partajează kernel-ul gazdei
CSP	Content Security Policy — header HTTP care restricționează sursele de conținut admis
Endpoint	URL expus de API care răspunde unei cereri HTTP specifice
Frontend	Componenta client-side, rulează în browser
HSTS	HTTP Strict Transport Security — forțează browser-ul să folosească HTTPS
IDOR	Insecure Direct Object Reference — vulnerabilitate OWASP A01
Idempotență	Proprietate: repetarea unei operații produce același efect ca aplicarea ei o singură dată
Lock pesimist	SELECT ... FOR UPDATE — blochează rândurile selectate până la COMMIT/ROLLBACK
Middleware	Strat de cod care interceptează cererile între recepție și procesare
Pydantic	Bibliotecă Python pentru validare date și serializare via anotări de tip
Reverse proxy	Server intermediar care primește cereri și le redirecționează către backend
SaaS	Software as a Service (ex: Stripe)
Salt	Șir aleator adăugat parolei înainte de hash, unic per utilizator
Stateless	Arhitectură în care serverul nu păstrează stare între cereri
Webhook	Callback HTTP trimis de un serviciu extern pentru a notifica un eveniment